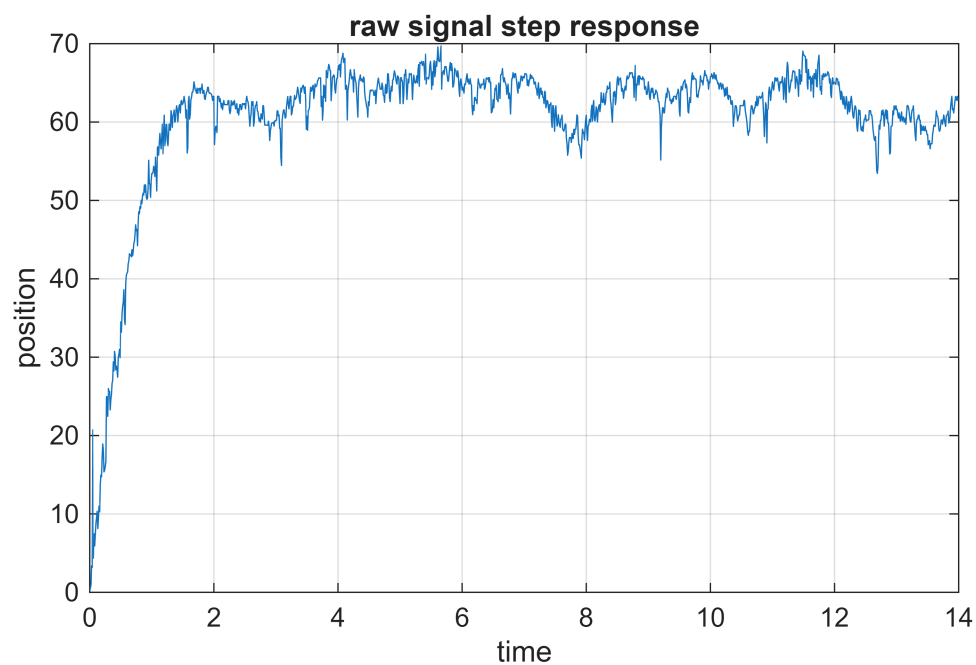


After getting the transfer function, we should definitely perform the analysis to our system on MATLAB :

step input response (150 on PWM) :

```
t=0:0.01:13.99;  
Ts=0.01;  
plot(t,raw_signal)  
grid on;  
title('raw signal step response')  
ylabel('position')  
xlabel('time')
```



S-domain TF :

```
num=[9.677e13,1.808e15];  
den=[1,1.804e5,4.822e14,2.645e15,4.291e15];  
G_s = tf(num,den)
```

G_s =

$$\frac{9.677e13 s + 1.808e15}{s^4 + 180400 s^3 + 4.822e14 s^2 + 2.645e15 s + 4.291e15}$$

Continuous-time transfer function.
Model Properties

Z-domain TF :

```
G_z= c2d(G_s, Ts, 'zoh')
```

```
G_z =
```

$$\frac{0.002137 z^3 - 0.001772 z^2 - 7.108e-11 z - 4.386e-30}{z^4 - 1.946 z^3 + 0.9466 z^2 + 1.585e-18 z + 4.637e-37}$$

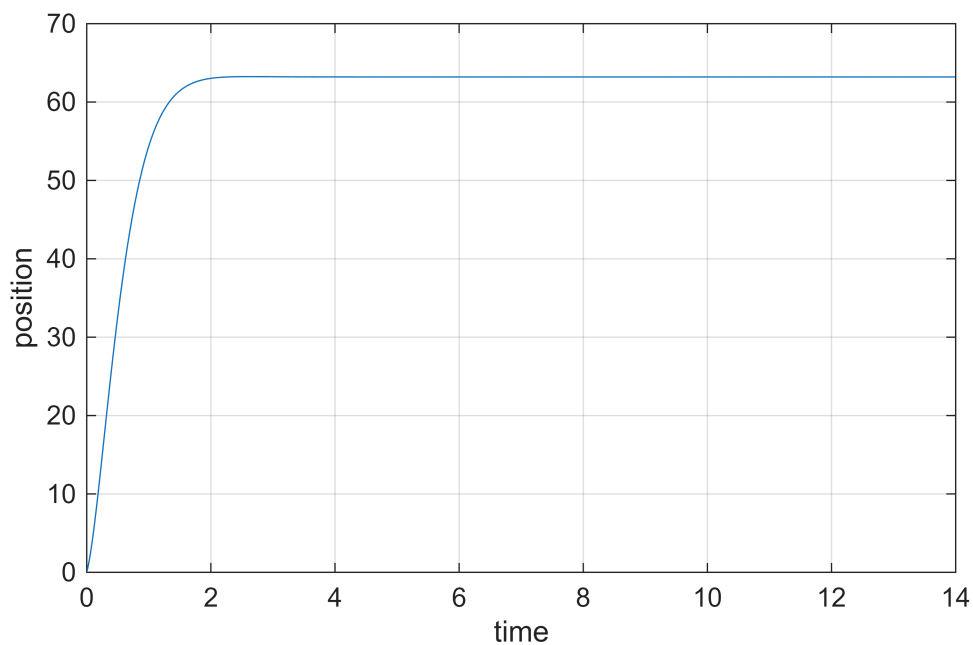
Sample time: 0.01 seconds

Discrete-time transfer function.

Model Properties

srep responses :

```
v=150; %(Volt step input gain);  
h=60;  %(target positon)  
[y,t] = step(v*G_z, t);  
plot(t,y)  
hold on  
grid on  
ylabel('position')  
xlabel('time')  
hold off
```



transient behaviour before the controller :

```
stepinfo(v*G_z)
```

```
ans = struct with fields:  
    RiseTime: 0.9900  
    TransientTime: 1.5900  
    SettlingTime: 1.5900  
    SettlingMin: 56.9869  
    SettlingMax: 63.2436
```

Overshoot: 0.0657
Undershoot: 0
Peak: 63.2436
PeakTime: 2.6100

PID tuner :

```
pidTuner(G_s, 'pid');
```

tuned PID in both s and z domain :

```
% Auto tune PID  
[C, info] = pidtune(G_s, 'PID');  
  
% Display gains  
Kp = C.Kp
```

```
Kp =  
4.0013
```

```
Ki = C.Ki
```

```
Ki =  
8.7888
```

```
Kd = C.Kd
```

```
Kd =  
0
```

```
PID_s=tf([Kd Kp Ki],[1 0])
```

```
PID_s =  
  
4.001 s + 8.789  
-----  
s
```

Continuous-time transfer function.
Model Properties

```
PID_z=c2d(PID_s, Ts, 'tustin')
```

```
PID_z =  
  
4.045 z - 3.957  
-----  
z - 1
```

Sample time: 0.01 seconds
Discrete-time transfer function.
Model Properties

Cascaded TF after PID in Z Domain :

```
cascaded_TF_z = series(G_z, PID_z)
```

```
cascaded_TF_z =
```

$$\frac{0.008643 z^4 - 0.01562 z^3 + 0.007011 z^2 + 2.813e-10 z + 1.736e-29}{z^5 - 2.946 z^4 + 2.892 z^3 - 0.9466 z^2 - 1.585e-18 z - 4.637e-37}$$

Sample time: 0.01 seconds

Discrete-time transfer function.

Model Properties

Closed-loop transfer function in Z Domain :

```
closed_loop_TF_z = feedback(cascaded_TF_z, 1)
```

```
closed_loop_TF_z =
```

$$\frac{0.008643 z^4 - 0.01562 z^3 + 0.007011 z^2 + 2.813e-10 z + 1.736e-29}{z^5 - 2.937 z^4 + 2.877 z^3 - 0.9396 z^2 + 2.813e-10 z + 1.736e-29}$$

Sample time: 0.01 seconds

Discrete-time transfer function.

Model Properties

Comparison between Transient Response before and after our designed PID :

```
transient_response_before=stepinfo(v*G_z) %before
```

```
transient_response_before = struct with fields:
```

```
    RiseTime: 0.9900
  TransientTime: 1.5900
    SettlingTime: 1.5900
    SettlingMin: 56.9869
    SettlingMax: 63.2436
    Overshoot: 0.0657
    Undershoot: 0
        Peak: 63.2436
    PeakTime: 2.6100
```

```
transient_response_after=stepinfo(h*closed_loop_TF_z) %after
```

```
transient_response_after = struct with fields:
```

```
    RiseTime: 0.4300
  TransientTime: 1.8900
    SettlingTime: 1.8900
    SettlingMin: 54.0206
    SettlingMax: 65.6650
    Overshoot: 9.4417
    Undershoot: 0
        Peak: 65.6650
    PeakTime: 0.8700
```

Comparison between the step steady state error before and after our designed PID :

```
%before
[y1, t1]=step(v*G_z);
ss_error_before=abs(h - y1(end) )
```

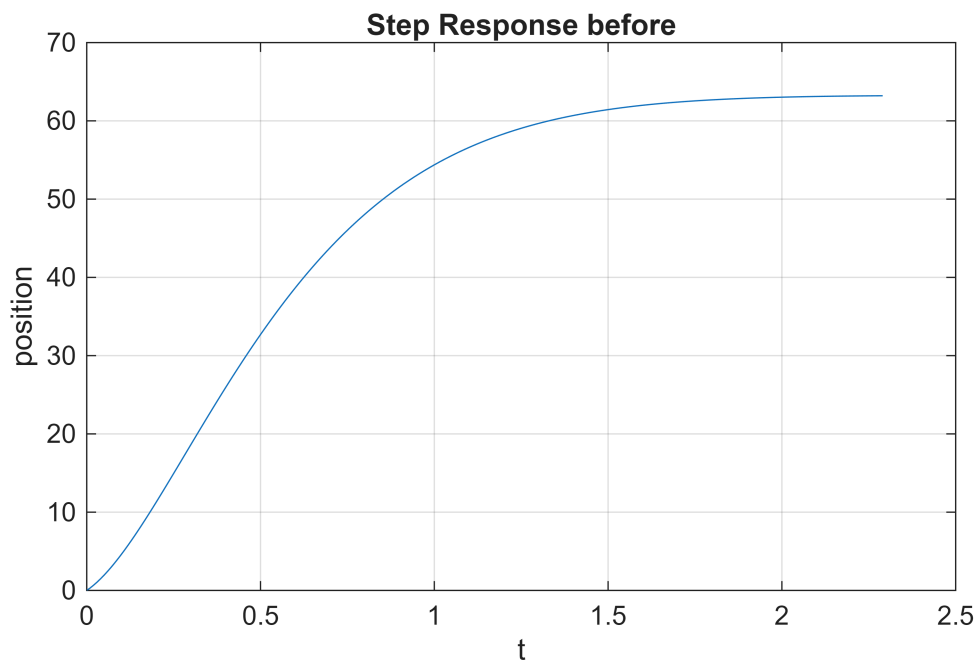
```
ss_error_before =
3.2080
```

```
%after
[y2, t2]=step(h*closed_loop_TF_z);
ss_error_after=abs(h - y2(end))
```

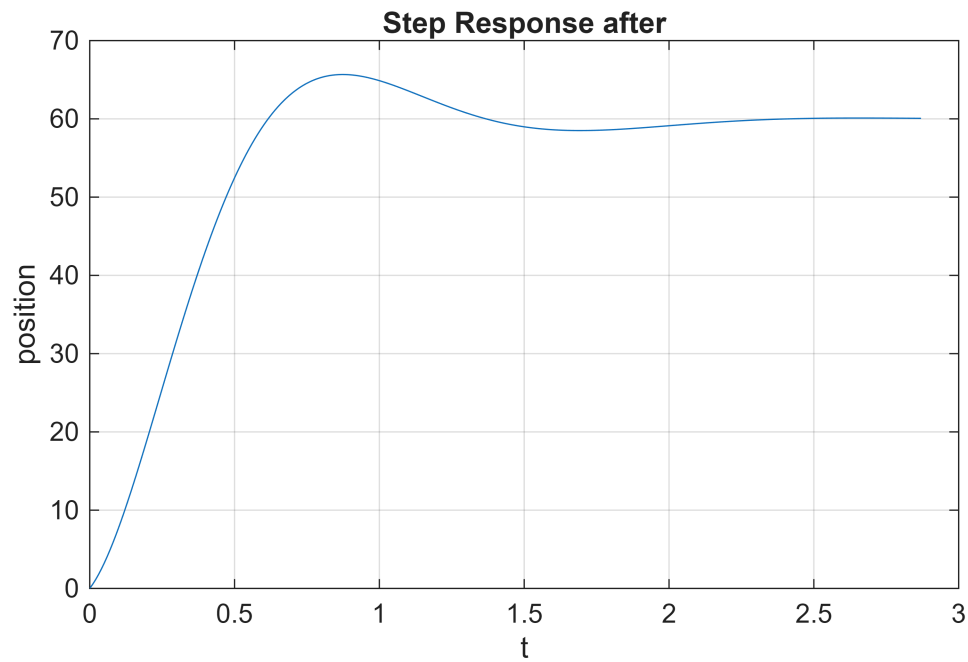
```
ss_error_after =
0.0620
```

Step response comparison :

```
[y1,t1]=step(v*G_z);
plot(t1,y1)
title('Step Response before');
xlabel('t');
ylabel('position')
grid on
```

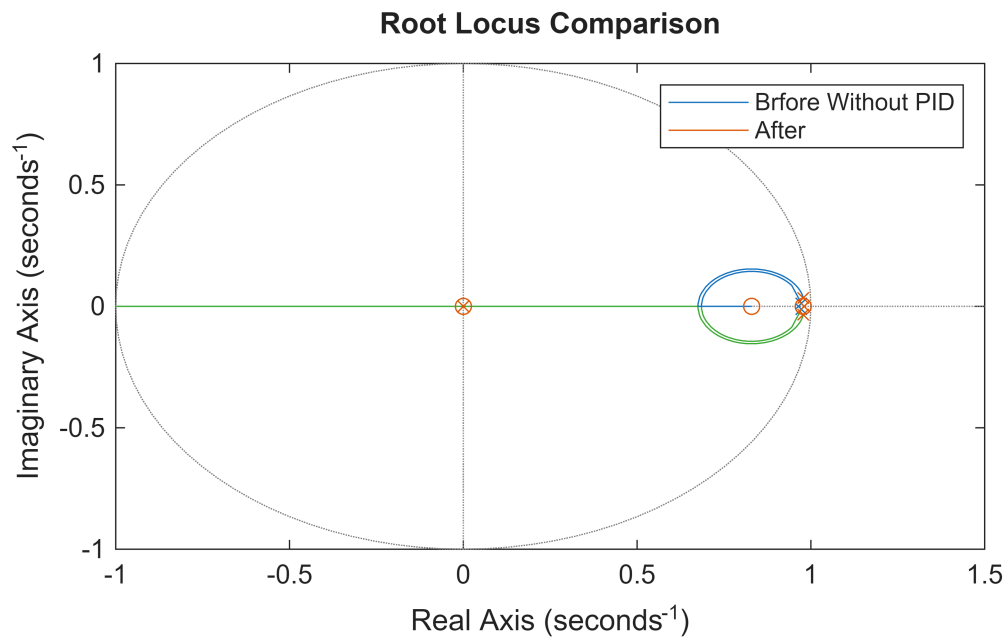


```
[y2, t2]=step(h*closed_loop_TF_z);
plot(t2,y2)
title('Step Response after');
xlabel('t');
ylabel('position')
grid on
```



Root Locus in Z-domain comparison (before and after inserting PID) :

```
figure;
rlocus(G_z)
hold on;
rlocus(closed_loop_TF_z);
legend('Before Without PID', 'After');
title('Root Locus Comparison');
hold off
```



```
isstable(G_z)    %before
```

```
ans = logical
      1
```

```
isstable(closed_loop_TF_z)    %after
```

```
ans = logical
      1
```